

Complete SonarQube Metrics Guide with Tiers

PART 1: MAINTAINABILITY METRICS

Tier 1: Critical - Must Track

1. **Technical Debt Ratio** [Use SonarQube built in TD ratio](#)

What It Measures:

- Ratio of technical debt (time to fix all issues) to total development time
- Expressed as percentage or days

Why Critical:

- Direct measure of code maintainability burden
- Predicts future development velocity
- Shows if debt is accumulating or being paid down

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=sqale_index,ncloc

Logic:

1. Get sqale_index (technical debt in minutes)
2. Get ncloc (lines of code)
3. Calculate: $(sqale_index / (ncloc \times development_cost_per_line)) \times 100$
- Default: 30 minutes per line of code
4. Technical Debt Ratio = $(debt / total_dev_cost) \times 100$

Alert Thresholds:

- <5% = Excellent (A rating)
- 5-10% = Good (B rating)
- 10-20% = Moderate (C rating)
- 20-50% = High (D rating)
- 50% = Critical (E rating) - IMMEDIATE ACTION

Trend to Watch:

- Increasing ratio = debt accumulating faster than being fixed
 - Track month-over-month change
-

2. Maintainability Rating

What It Measures:

- Overall grade (A-E) based on technical debt ratio
- Aggregated view of code health

Why Critical:

- Single metric for executive dashboards
- Directly impacts development velocity
- Predicts maintenance costs

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=sqale_rating

Logic:

1. Fetch sqale_rating value (1-5)
2. Map to letter grades:
 - 1 = A (debt ratio ≤5%)
 - 2 = B (debt ratio 6-10%)
 - 3 = C (debt ratio 11-20%)
 - 4 = D (debt ratio 21-50%)
 - 5 = E (debt ratio >50%)

Alert Thresholds:

- C or worse = needs attention
- D or E = CRITICAL - stop new features, focus on cleanup

Correlation:

- Track with GitLab "Bug vs Feature Ratio"
 - High bug ratio + low maintainability rating = systemic quality issues
-

3. Code Smells (Total Count)

What It Measures:

- Design and style issues that don't break code but increase maintenance burden
- Examples: long methods, deep nesting, magic numbers, duplicated logic

Why Critical:

- Leading indicator of future bugs
- Slows down feature development
- Makes onboarding harder

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=code_smells

Logic:

1. Fetch total code_smells count
2. Track by severity:
 - Blocker (0 - highest priority)
 - Critical
 - Major
 - Minor
 - Info (5 - lowest priority)

Alert Thresholds:

- <50 per 10k lines = Good
- 50-100 per 10k lines = Acceptable
- 100 per 10k lines = High debt

- ANY blocker/critical smells = immediate fix

Trend to Watch:

- New code smells per week
- If new smells > fixed smells for 3+ weeks = debt accumulating

4. Code Smells - New Code

What It Measures:

- Code smells introduced in recent changes (last 30 days)
- Measures quality of new development

Why Critical:

- Shows if team is adding to debt or maintaining quality
- More actionable than total debt
- Easier to fix fresh issues

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=new_code_smells

Logic:

1. Fetch new_code_smells (smells added since last version/branch)
2. Track daily/weekly
3. Compare: new smells added vs old smells fixed

Alert Thresholds:

- 0 new smells in release = Excellent
- <5 new smells in release = Good
- 10 new smells in release = WARNING
- New smells > fixed smells for 2+ weeks = RED ALERT

Best Practice:

- Set Quality Gate to fail if new code adds smells
- Block merges that increase debt

Tier 2: Important - Should Track

5. Cyclomatic Complexity

What It Measures:

- Number of independent execution paths through code
- Count of decision points (if, while, for, case, etc.)

Why Important:

- Complex code is harder to test
- More paths = more potential bugs
- Difficult to understand and modify

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=complexity,functions

Logic:

1. Get total complexity score
2. Get number of functions
3. Calculate average: complexity / functions
4. Identify high-complexity methods via:

GET

/api/measures/component_tree?component=:project_key&metricKeys=complexity&ps=100

Alert Thresholds:

- Function complexity <10 = Simple
- Function complexity 10-20 = Moderate
- Function complexity 20-50 = Complex - needs refactoring
- Function complexity >50 = Critical - immediate refactoring

Trend to Watch:

- Average complexity increasing = code getting harder to maintain
- Track functions with complexity >20

6. Cognitive Complexity

What It Measures:

- How difficult code is to understand (harder than cyclomatic)
- Accounts for nested structures and break statements
- More human-focused metric

Why Important:

- Better predictor of maintainability than cyclomatic
- Directly relates to developer productivity
- Identifies truly confusing code

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=cognitive_complexity

Logic:

1. Fetch cognitive_complexity score
2. Calculate per function/method
3. Flag functions with high cognitive load

Alert Thresholds:

- Cognitive complexity <15 = Good
- 15-25 = Review needed
- 25 = Refactor immediately

Correlation:

- High cognitive complexity + high bug count = strong refactoring candidate
-

7. Duplicated Code Percentage

What It Measures:

- Percentage of duplicated lines across codebase
- SonarQube detects copy-paste patterns

Why Important:

- Duplicated bugs = fix needed in multiple places
- Higher maintenance burden
- Violates DRY principle

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=duplicated_lines_density

Logic:

1. Fetch duplicated_lines_density (percentage)
2. Also track:
 - duplicated_lines (absolute count)
 - duplicated_blocks (number of duplicate blocks)
 - duplicated_files (files containing duplicates)

Alert Thresholds:

- <3% duplication = Excellent
- 3-5% = Acceptable
- 5-10% = High - refactoring recommended
- 10% = Critical - serious maintainability issue

Trend to Watch:

- Duplication increasing = copy-paste coding pattern
 - Track new duplication in recent code
-

8. Duplicated Lines in New Code

What It Measures:

- Duplication introduced in recent changes
- Measures discipline in new development

How to Calculate from SonarQube API:

GET

/api/measures/component?component=:project_key&metricKeys=new_duplicated_lines_density

Logic:

1. Track duplication percentage in new/changed code
2. Should be 0% in ideal case

Target:

- 0% new duplication (enforce in Quality Gate)
-

Tier 3: Nice to Have - Context

9. Comment Density (Documentation)

What It Measures:

- Ratio of comment lines to code lines

- Indicates code documentation level

Why Track:

- Under-documented code = harder onboarding
- Over-commented = potentially outdated/redundant comments
- Balance is key

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=comment_lines_density

Logic:

1. Fetch comment_lines_density (percentage)
2. Also get:
 - comment_lines (absolute count)
 - ncloc (non-comment lines of code)

Target Range:

- 10-20% comment density = Good balance
- <5% = Under-documented
- 30% = Over-commented (possibly outdated)

10. Lines of Code (NCLOC)

What It Measures:

- Total non-comment lines of code
- Codebase size metric

Why Track:

- Normalize other metrics (per 1000 lines)
- Track codebase growth
- Estimate maintenance costs

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=ncloc

Logic:

1. Track ncloc over time
2. Calculate growth rate
3. Use to normalize: bugs per 1k lines, smells per 1k lines

Watch For:

- Rapid growth without proportional team growth = quality risk
-

11. File Complexity Distribution

What It Measures:

- How complexity is distributed across files
- Identifies hotspot files

How to Calculate from SonarQube API:

GET

/api/measures/component_tree?component=:project_key&metricKeys=complexity&strategy=leaves

Logic:

1. Get complexity per file
2. Sort descending
3. Identify top 10 most complex files
4. Track if same files appear repeatedly

Action:

- Top 10% of files by complexity = refactoring candidates
-
-

PART 2: RELIABILITY METRICS

Tier 1: Critical - Must Track

12. Bugs (Total Count)

What It Measures:

- Code defects that will likely cause runtime failures
- Examples: null pointer dereferences, resource leaks, logic errors

Why Critical:

- Direct impact on software correctness
- User-facing failures
- Production incidents

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=bugs

Logic:

1. Fetch total bugs count
2. Break down by severity:
 - Blocker
 - Critical
 - Major
 - Minor
 - Info

Alert Thresholds:

- 0 blocker/critical bugs = Good
- ANY blocker bug = IMMEDIATE FIX
- 5 critical bugs = RED ALERT

- Bugs per 1k lines >1 = quality issue

Trend to Watch:

- New bugs introduced per release
- Bug fix rate vs introduction rate

13. Reliability Rating

What It Measures:

- Overall grade (A-E) based on bug count and severity

- Aggregated reliability health

Why Critical:

- Executive-level quality indicator
- Predicts production stability
- User experience proxy

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=reliability_rating

Logic:

1. Fetch reliability_rating (1-5)
2. Map to grades:
 - A (1) = 0 bugs
 - B (2) = At least 1 minor bug
 - C (3) = At least 1 major bug
 - D (4) = At least 1 critical bug
 - E (5) = At least 1 blocker bug

Alert Thresholds:

- A-B = Acceptable
- C = Warning
- D-E = CRITICAL - block releases

Correlation:

- Track with GitLab "Issue Reopen Rate"
 - High reopen rate + low reliability rating = poor testing
-

14. Bugs in New Code

What It Measures:

- Bugs introduced in recent changes
- Quality of new development

Why Critical:

- Shows if team is adding to defect backlog
- Fresh bugs easier to fix (context still fresh)
- Leading indicator of code review effectiveness

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=new_bugs

Logic:

1. Fetch new_bugs (added since last version)
2. Track per commit, per MR, per sprint
3. Compare with bugs fixed

Alert Thresholds:

- 0 new bugs = Excellent
- 1-2 new bugs in release = Acceptable
- 3 new bugs = WARNING

- New bugs > fixed bugs for 2+ sprints = RED ALERT

Best Practice:

- Quality Gate: fail if new code introduces any bugs
 - Track in GitLab pipeline
-

Tier 2: Important - Should Track

15. Reliability Remediation Effort

What It Measures:

- Estimated time to fix all reliability issues
- Expressed in hours/days

Why Important:

- Resource planning for bug fixes
- Prioritization of technical debt

How to Calculate from SonarQube API:

GET
/api/measures/component?component=:project_key&metricKeys=reliability_remediation_effort

Logic:

1. Fetch reliability_remediation_effort (minutes)
2. Convert to hours/days
3. Track trend over time

Use Case:

- Sprint planning: allocate X hours for bug fixes
 - Track if effort increasing or decreasing
-
-

PART 3: SECURITY METRICS

Tier 1: Critical - Must Track

16. Vulnerabilities (Total Count)

What It Measures:

- Known security weaknesses in code
- Examples: SQL injection, XSS, insecure crypto, hard-coded secrets

Why Critical:

- Direct path to security breaches
- Compliance/regulatory risk
- Reputation damage if exploited

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=vulnerabilities

Logic:

1. Fetch total vulnerabilities
2. Break down by severity:
 - Blocker (e.g., SQL injection)
 - Critical (e.g., weak encryption)
 - Major
 - Minor
 - Info

Alert Thresholds:

- 0 blocker/critical vulnerabilities = Good
- ANY blocker vulnerability = DROP EVERYTHING, FIX NOW
- 2 critical vulnerabilities = RED ALERT

- 10 total vulnerabilities = security review needed

Trend to Watch:

- New vulnerabilities per release
 - Time to fix vulnerabilities (should be <24 hours for critical)
-

17. Security Rating

What It Measures:

- Overall security grade (A-E)
- Based on vulnerability count and severity

Why Critical:

- Executive dashboard metric
- Compliance reporting
- Risk assessment

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=security_rating

Logic:

1. Fetch security_rating (1-5)
2. Map to grades:
 - A (1) = 0 vulnerabilities
 - B (2) = At least 1 minor vulnerability
 - C (3) = At least 1 major vulnerability
 - D (4) = At least 1 critical vulnerability
 - E (5) = At least 1 blocker vulnerability

Alert Thresholds:

- A-B = Acceptable
- C = Warning - security review
- D-E = CRITICAL - block deployments

Compliance:

- Many standards require A or B rating for production
-

18. Vulnerabilities in New Code

What It Measures:

- Security issues introduced in recent changes
- Quality of security practices in development

Why Critical:

- Fresh vulnerabilities easier to fix
- Shows security awareness in team
- Prevent security debt accumulation

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=new_vulnerabilities

Logic:

1. Fetch new_vulnerabilities count
2. Track per commit/MR
3. Should be 0

Alert Thresholds:

- 0 new vulnerabilities = Excellent (ONLY acceptable target)
- ANY new vulnerability = FAIL build, block merge

Best Practice:

- Quality Gate: absolutely 0 tolerance for new vulnerabilities
 - Require security review for any violation
-

19. Security Hotspots

What It Measures:

- Code sections requiring manual security review
- Potential risks that need human judgment
- Examples: authentication logic, file uploads, data encryption

Why Critical:

- Automated tools can't determine if implementation is secure
- Requires security expert review
- Unreviewed hotspots = unknown risk

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=security_hotspots

Logic:

1. Fetch total security_hotspots count
2. Track review status:
 - To Review
 - Reviewed (Safe/Fixed)

Alert Thresholds:

- 100% reviewed = Good
- <80% reviewed = WARNING
- <50% reviewed = RED ALERT

Best Practice:

- Require all hotspots reviewed before production deployment
 - Assign security champion for reviews
-

20. Security Hotspots Reviewed

What It Measures:

- Percentage of hotspots that have been manually reviewed
- Shows security diligence

How to Calculate from SonarQube API:

GET

/api/measures/component?component=:project_key&metricKeys=security_hotspots_reviewed

Logic:

1. Fetch security_hotspots_reviewed (percentage)
2. Track trend over time

Target:

- 100% reviewed before production
-

Tier 2: Important - Should Track

21. Security Review Rating

What It Measures:

- Grade (A-E) based on hotspot review percentage
- Shows security review diligence

How to Calculate from SonarQube API:

GET

/api/v4/measures/component?component=:project_key&metricKeys=security_review_rating

Logic:

1. Based on % of hotspots reviewed:
 - A = 100% reviewed
 - B = 80-99% reviewed
 - C = 70-79% reviewed
 - D = 50-69% reviewed
 - E = <50% reviewed

Target:

- A rating (100% reviewed)
-

22. Security Remediation Effort

What It Measures:

- Estimated time to fix all security issues

- Resource planning metric

How to Calculate from SonarQube API:

GET

/api/measures/component?component=:project_key&metricKeys=security_remediation_effort

Logic:

1. Fetch security_remediation_effort (minutes)
2. Convert to hours/days
3. Prioritize by vulnerability severity

Use Case:

- Sprint planning for security fixes
 - Risk assessment (high effort = high risk exposure time)
-
-

PART 4: TEST COVERAGE METRICS

Tier 1: Critical - Must Track

23. Test Coverage (Overall)

What It Measures:

- Percentage of code executed by automated tests
- Lines/branches covered by unit/integration tests

Why Critical:

- Low coverage = high risk of undetected bugs
- Direct correlation with production defects
- Safety net for refactoring

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=coverage

Logic:

1. Fetch coverage percentage

2. Also track:

- line_coverage (% of lines covered)
- branch_coverage (% of branches covered)

Alert Thresholds:

- 80% = Good
- 60-80% = Acceptable
- 40-60% = WARNING
- <40% = RED ALERT (very high risk)

Trend to Watch:

- Coverage declining = quality degrading
- Track month-over-month change

Correlation:

- Low coverage + high bug count = testing gap
 - Compare with GitLab "Test Failure Rate"
-

24. Coverage on New Code

What It Measures:

- Test coverage of recently added/changed code
- Shows testing discipline in new development

Why Critical:

- More important than overall coverage
- Prevents further coverage degradation
- Easier to test new code than legacy

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=new_coverage

Logic:

1. Fetch new_coverage (coverage of new/changed code)
2. Track per commit, per MR

3. Should be higher than overall coverage

Alert Thresholds:

- 80% = Excellent (enforce in Quality Gate)
- 60-80% = Acceptable
- <60% = FAIL - block merge

Best Practice:

- Quality Gate: require >80% coverage on new code
 - Higher bar for new code than legacy
-

Tier 2: Important - Should Track

25. Line Coverage

What It Measures:

- Percentage of executable lines hit by tests
- More granular than overall coverage

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=line_coverage

Logic:

1. Fetch line_coverage
2. Also track:
 - lines_to_cover (total executable lines)
 - uncovered_lines (lines not covered)

Use Case:

- Identify specific uncovered lines
 - Targeted test improvement
-

26. Branch Coverage

What It Measures:

- Percentage of code branches (if/else paths) covered by tests
- More stringent than line coverage

Why Important:

- Ensures all logic paths tested
- Catches edge cases
- Better quality indicator than line coverage alone

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=branch_coverage

Logic:

1. Fetch branch_coverage
2. Also track:
 - conditions_to_cover (total branches)
 - uncovered_conditions (branches not covered)

Target:

- 70% branch coverage = Good
 - Should be slightly lower than line coverage (harder to achieve)
-

27. Condition Coverage

What It Measures:

- Coverage of boolean conditions in expressions
- Most granular coverage metric

How to Calculate from SonarQube API:

GET

/api/measures/component?component=:project_key&metricKeys=conditions_to_cover,uncovered_conditions

Logic:

1. Calculate: $(\text{conditions_to_cover} - \text{uncovered_conditions}) / \text{conditions_to_cover} \times 100$

Tier 3: Nice to Have - Context

28. Unit Test Count

What It Measures:

- Total number of unit tests
- Growth of test suite

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=tests

Logic:

1. Track test count over time
2. Calculate tests per 1000 lines of code

Watch For:

- Tests growing proportionally with code

29. Test Execution Time

What It Measures:

- Time to run full test suite
- CI/CD feedback loop speed

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=test_execution_time

Logic:

1. Track test_execution_time (milliseconds)
2. Monitor trend

Watch For:

- Increasing test time = CI/CD slowdown
- Target: <10 minutes for fast feedback

30. Skipped Tests

What It Measures:

- Tests marked as skipped/ignored
- Potential quality blind spots

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=skipped_tests

Logic:

1. Count skipped tests
2. Investigate why skipped

Alert:

- Should be 0 (skipped tests = untested code)
-
-

PART 5: QUALITY GATE METRICS

Tier 1: Critical - Must Track

31. Quality Gate Status

What It Measures:

- Pass/Fail status of overall quality gate
- Combines multiple metrics into single decision

Why Critical:

- Single source of truth for "Is code ready?"
- Prevents bad code from reaching production
- Enforces quality standards

How to Calculate from SonarQube API:

GET /api/qualitygates/project_status?projectKey=:project_key

Logic:

1. Fetch overall status: PASSED or FAILED
2. Get conditions that failed
3. Track pass rate over time

Standard Quality Gate Conditions:

- Coverage on new code $\geq 80\%$
- Duplicated lines on new code $\leq 3\%$
- Maintainability rating on new code = A
- Reliability rating on new code = A
- Security rating on new code = A
- Security hotspots reviewed = 100%

Alert Thresholds:

- FAILED gate = block merge/deployment
- Track failure reasons for patterns

Best Practice:

- Integrate with GitLab CI/CD
- Block merges if Quality Gate fails

32. Quality Gate Pass Rate

What It Measures:

- Percentage of builds/branches passing Quality Gate
- Team quality discipline

How to Calculate from SonarQube API:

Logic:

1. Track Quality Gate status per analysis
2. Calculate: $(\text{passed} / \text{total analyses}) \times 100$
3. Track weekly/monthly trend

Target:

- 95% pass rate = Good discipline
 - <80% pass rate = standards too strict or team needs training
-

Tier 2: Important - Should Track

33. Quality Gate Conditions Met

What It Measures:

- Which specific conditions passed/failed
- Breakdown of quality issues

How to Calculate from SonarQube API:

GET /api/qualitygates/project_status?projectKey=:project_key

Logic:

1. Parse conditions array
2. Track which conditions fail most often:
 - Coverage failures
 - Duplication failures
 - Security failures
 - etc.

Use Case:

- Identify recurring quality patterns
 - Target training/tooling improvements
-
-

PART 6: HISTORICAL & TREND METRICS

Tier 1: Critical - Must Track

34. Technical Debt Trend

What It Measures:

- Change in technical debt over time
- Shows if debt growing or shrinking

Why Critical:

- Predicts long-term maintainability
- Shows effectiveness of refactoring efforts
- Resource planning

How to Calculate from SonarQube API:

GET /api/measures/search_history?component=:project_key&metrics=sqale_index

Logic:

1. Fetch sqale_index (technical debt) over time
2. Calculate week-over-week, month-over-month change
3. Plot trend line

Alert Thresholds:

- Debt increasing >10% per month = RED ALERT
- Debt stable or decreasing = Good

Correlation:

- Compare with GitLab velocity
 - Increasing debt + declining velocity = systemic issue
-

35. New Technical Debt Added

What It Measures:

- Debt introduced in recent code
- Shows if team adding to or reducing debt

How to Calculate from SonarQube API:

GET /api/measures/component?component=:project_key&metricKeys=new_technical_debt

Logic:

1. Fetch new_technical_debt (minutes added in new code)
2. Compare with debt removed (via fixes)

3. Net change = new debt - removed debt

Target:

- Net debt should be negative or zero
 - Positive net debt for 3+ sprints = unsustainable
-

Tier 2: Important - Should Track

36. Bug Density Trend

What It Measures:

- Bugs per 1000 lines of code over time
- Normalizes bug count by codebase size

How to Calculate from SonarQube API:

GET /api/measures/search_history?component=:project_key&metrics=bugs,ncloc

Logic:

1. Fetch bugs and ncloc over time
2. Calculate: $(\text{bugs} / \text{ncloc}) \times 1000$
3. Track trend

Target:

- Density declining = quality improving
-

37. Coverage Trend

What It Measures:

- Test coverage percentage over time
- Shows testing discipline consistency

How to Calculate from SonarQube API:

GET /api/measures/search_history?component=:project_key&metrics=coverage

Logic:

1. Track coverage weekly/monthly
2. Identify upward or downward trend

Watch For:

- Declining coverage = testing discipline eroding
-
-

PART 7: COMPONENT-LEVEL METRICS

Tier 2: Important - Should Track

38. Hotspot Files (Worst Offenders)

What It Measures:

- Files with most issues (bugs + code smells + vulnerabilities)
- Identifies refactoring priorities

How to Calculate from SonarQube API:

GET

/api/measures/component_tree?component=:project_key&metricKeys=bugs,code_smells,vulnerabilities&strategy=leaves

Logic:

1. Get metrics for each file
2. Sum: bugs + code_smells + vulnerabilities per file
3. Sort descending
4. Identify top 10 worst files

Action:

- Top 10 files = priority refactoring targets
 - Track if same files appear repeatedly (chronic issues)
-

39. Most Complex Methods/Functions

What It Measures:

- Functions with highest cyclomatic/cognitive complexity
- Specific refactoring candidates

How to Calculate from SonarQube API:

GET

/api/measures/component_tree?component=:project_key&metricKeys=cognitive_complexity&qualifiers=FILE&ps=500

Logic:

1. Get complexity per method/function
2. Sort descending
3. Flag methods with complexity >15

Action:

- Refactor top 10 most complex methods
-
-

SONARQUBE IMPLEMENTATION PRIORITY

Phase 1: Essential Quality Dashboard (Week 1)

Must track these 10 metrics first:

1. **Quality Gate Status** - Pass/Fail
2. **Technical Debt Ratio** - Overall maintainability
3. **Maintainability Rating** - A-E grade
4. **Bugs (Total)** - Reliability health
5. **Reliability Rating** - A-E grade
6. **Vulnerabilities (Total)** - Security health
7. **Security Rating** - A-E grade
8. **Test Coverage** - Overall safety net
9. **Code Smells** - Maintainability backlog
10. **Duplicated Code %** - Redundancy level

Phase 2: New Code Quality (Week 2)

Add these to enforce quality in new development:

- 11. Code Smells in New Code
- 12. Bugs in New Code
- 13. Vulnerabilities in New Code
- 14. Coverage on New Code
- 15. Duplicated Lines in New Code
- 16. New Technical Debt Added

Phase 3: Deep Analysis (Month 2)

Add for detailed insights:

- 17. Cyclomatic Complexity
- 18. Cognitive Complexity
- 19. Security Hotspots & Review Status
- 20. Branch Coverage
- 21. Hotspot Files (Worst Offenders)
- 22. Technical Debt Trend

Phase 4: Advanced Metrics (Month 3+)

Context and optimization:

- 23. Bug Density Trend
- 24. Coverage Trend
- 25. Most Complex Methods
- 26. Test Execution Time
- 27. Quality Gate Pass Rate



SONARQUBE + GITLAB CORRELATION MATRIX

Critical Correlations to Track:

SonarQube Metric	GitLab Metric	What It Reveals
Technical Debt Ratio	Cycle N	Time High debt = slower development Bugs in New Code Issue Reopen Rate Poor code quality = rework Coverage on New Code MR Revert Rate Low coverage = bugs escaping Security Rating Deployment Failure Rate Security issues = production problems Code Smells MR Merge Time Complex code = slower reviews Maintainability Rating Sprint Velocity Low maintainability = declining velocity Quality Gate Failures Pipeline Success Rate Quality issues = failed builds Duplicated Code Code Churn High duplication = repeated changes

ALERT MATRIX

RED ALERTS (Drop Everything):

- Quality Gate: FAILED
- Security Rating: D or E
- ANY blocker vulnerability
- ANY blocker bug
- Maintainability Rating: E
- Coverage on new code: <40%

WARNINGS (Address This Sprint):

- Technical Debt increasing >10%/month
- 5 critical bugs
- 2 critical vulnerabilities
- Security hotspots <80% reviewed
- Coverage trending down 3+ months
- Code smells in new code >10/week

WATCH (Monitor Trend):

- Maintainability Rating: C
 - Duplication >5%
 - Cognitive complexity increasing
 - Test execution time growing
-



SONARQUBE API QUICK REFERENCE

Most Used Endpoints:

Overall project metrics

GET /api/measures/component?component=:project_key&metricKeys=:metrics

Historical data

GET /api/measures/search_history?component=:project_key&metrics=:metrics

Quality Gate status

GET /api/qualitygates/project_status?projectKey=:project_key

Component tree (files/modules)

GET /api/measures/component_tree?component=:project_key&metricKeys=:metrics

Issues (bugs, vulnerabilities, code smells)

GET /api/issues/search?componentKeys=:project_key&types=:issue_types

Security hotspots

GET /api/hotspots/search?projectKey=:project_key

Key Metric Keys:

Maintainability

sqale_index, sqale_rating, code_smells, new_code_smells, complexity, cognitive_complexity, duplicated_lines_density, new_duplicated_lines_density, comment_lines_density, ncloc

Reliability

bugs, new_bugs, reliability_rating, reliability_remediation_effort

Security

vulnerabilities, new_vulnerabilities, security_rating,
security_hotspots, security_hotspots_reviewed,
security_review_rating, security_remediation_effort

Coverage

coverage, new_coverage, line_coverage, branch_coverage,
lines_to_cover, uncovered_lines, conditions_to_cover,
uncovered_conditions, tests, test_execution_time

Quality Gate

alert_status (Quality Gate status)